Agent Mesh SRE

Kafka Operations
Orchestrated by AI Agents

Monitor · Reason · Act · Learn

Confluent for Kubernetes on DigitalOcean · KRaft mode · 3 brokers · 7 topics · Deployed on Vercel

STRATEGIC PIVOT

From Kafka Ops POC to Agentic AI Data Platform

This POC demonstrates a pattern that extends well beyond operations — Confluent as the intelligent backbone for IBM's Agentic AI applications.

Kafka Operations Tool	→	**Real-Time AI Data Pipeline**	Confluent streams live enterprise data — transactions, events, logs, sensor feeds — directly into AI agent context windows as it happens.
Human-in-the-Loop Approvals	→	**Policy-Governed AI Actions**	The same approval gate pattern governs any AI-initiated action in production — financial moves, infra changes, customer communications.
Self-Healing Kafka Cluster	→	**Self-Healing AI Infrastructure**	Agents that detect, reason, and remediate apply universally — across databases, APIs, microservices, and the AI agents themselves.

ARCHITECTURE VISION

Confluent - Realtime Data Engine for Agents

Every IBM enterprise application becomes a real-time data source. Every AI agent decision is governed, audited, and fed back as a learning event.

Confluent becomes the nervous system — data flows in, agents reason, decisions are gated, outcomes feed back as learning

IBM OPPORTUNITY

What we get from this POC

Three capabilities that enterprise AI deployments consistently lack — all demonstrated in this POC.

01

Real-Time Grounding for AI Agents

Most enterprise AI runs on stale batch data. Confluent streams live facts — inventory levels, transaction states, system telemetry — directly into agent reasoning loops as they happen. Agents make decisions on current reality, not yesterday's snapshot.

Applies to: watsonx, AIOps, IBM Financial Services AI

02

Governed, Auditable AI Actions

The policy gate pattern — where every consequential AI action requires explicit operator approval and is written to an immutable audit topic — addresses the compliance gap in enterprise AI deployments. Every decision has an actor, a timestamp, and a rationale on the record.

Applies to: regulated industries, IBM Consulting AI delivery

03

Self-Improving AI Infrastructure

The compacted lessons topic creates a persistent, growing knowledge base that agents read on every cycle. IBM deployments could use the same pattern to accumulate domain-specific learnings from production incidents — without retraining models.

Applies to: IBM SRE, enterprise IT automation, watsonx Orchestrate

The POC runs today at agent-mesh-sre.vercel.app · Kafka cluster live on DigitalOcean · Open for IBM evaluation

WHY NOW

The Agentic AI Inflection Point

Three forces converging in 2025-26 make this the right moment for IBM to establish the Confluent-backed Agentic AI pattern.

Enterprise AI is Production-Ready

IBM watsonx, Granite models, and AWS Bedrock have crossed the enterprise readiness threshold. The gap is no longer model capability — it is real-time grounding and governed execution.

What this POC adds:

Live Kafka stream as the real-time fact layer for agents. No batch ETL. No stale context.

Compliance is the Bottleneck

Regulated industries — banking, healthcare, government — cannot deploy AI without audit trails and human override. This is the single biggest blocker to IBM AI consulting revenue.

What this POC adds:

Policy gate pattern + 365-day immutable audit topic. Every AI decision has an actor, timestamp, and rationale. Built-in compliance.

Kafka is Already Everywhere

IBM's largest enterprise customers already run Confluent or IBM Event Streams. The streaming backbone exists. It just isn't connected to the AI layer yet.

What this POC adds:

A working reference architecture that bridges Kafka topics to AI agent workflows — deployable on any IBM client's existing Confluent cluster.

IBMer 2026 Bobathon

Agent Mesh SRE

Confluent as the Agentic AI Backbone for IBM

01

Working POC, not a proposal

Live at agent-mesh-sre.vercel.app with a real Confluent for Kubernetes cluster on DigitalOcean. 11 Kafka failure scenarios, 4 human-gated approvals, full audit trail.

02

Reference prototype for Confluent Smart Platform for AI strategy

Demonstrates Confluent as the streaming backbone for watsonx agents — real-time grounding, policy governance, self-improving knowledge via compacted topics.

03

Applicable to IBM's largest client verticals

Financial services, healthcare, government, telco — any regulated IBM client already running Kafka can adopt this pattern without greenfield infrastructure.

`agent-mesh-sre.vercel.app · github.com/surajcsibm/agent-mesh-sre-real · Kafka: 64.227.25.232:31090`

Two Roles, One Platform, Common problems:

Kafka incidents are slow to detect, harder to diagnose, and leave no audit trail.

K

Ken — Kafka Administrator

Platform SRE, owns cluster operations

Current workflow:

- Monitors consumer lag manually across topics
- Diagnoses controller failovers by reading broker logs
- No structured record of remediation actions taken
- On-call pages with metric thresholds but no context

With this POC:

Automated incident detection and root-cause analysis delivered via approval gate — with a full audit trail.

B

Ben — Kafka Developer

Owns a payments producer and consumer group

Current workflow:

- Producer ACK timeouts surface with no root cause
- Schema incompatibilities break consumers silently
- No topic health view outside of custom dashboards
- Depends on the ops team to investigate incidents

With this POC:

Producer timeouts and schema issues are detected and diagnosed by the Monitor agent. Notifications routed directly.

ARCHITECTURE

The M→R→A→L Loop

A continuous cycle running across four specialised agents, connected by Kafka topics.

MONITOR

Polls cluster metrics via Kafka Admin API

Detects anomalies against baselines

Raises incidents on ops.incidents.v1

→

REASON

Cross-correlates broker, consumer, and topic signals to identify root cause

Produces confidence-scored analysis

→

ACT

Selects and calls the appropriate MCP tool

Requires human approval for any operation

that mutates cluster state

→

LEARN

Records outcome, lag delta, and notes to ops.lessons.v1 (log-compacted)

Fed back into the next Monitor cycle

Lessons feed back into Monitor on the next cycle → the system refines its detection over time

AGENT DESIGN

Four Agents, Single Responsibility Each

I

Intake

MCP Gateway

Entry point for operator-initiated requests. Validates and routes to the mesh via ops.requests.v1. Exposes intake.submitOpsRequest.

```
Produces to:
ops.requests.v
1
```

M

Monitor

MRAL Coordinator

Drives the full M→R→A→L cycle. Consumes metrics, raises incidents, calls remediation tools, records lessons. The core agent.

```
Produces to:
ops.incidents.
v1
ops.actions.au
dit.v1
ops.lessons.v1
```

W

Writer

Postmortem Authoring

Drafts structured incident reports after scenario completion. Consumes from ops.incidents.v1. Tool: writer.draftIncidentReport.

```
Produces to:
ops.actions.au
dit.v1
```

N

Notification

Outbound Routing

Routes alerts to Slack, ITSM, and email on completion or escalation. Tools: notify.slack, itsm.openTicket.

```
Produces to:
ops.notificati
ons.v1
```

TOPIC DESIGN

Seven Topics — Purpose-Built Retention

Confluent for Kubernetes on DigitalOcean · KRaft mode · RF=3 · 64.227.25.232:31090

TOPIC	PARTS	RETENTION	CLEANUP	PURPOSE
ops.requests.v1	6	7 d	delete	Operator requests from Intake agent
ops.kafka.metrics.v1	6	1 d	delete	Cluster telemetry polled by Monitor
ops.incidents.v1	6	30 d	delete	Incidents raised by Monitor agent
ops.actions.audit.v1	12	365 d	delete	Full audit trail — every action and decision
ops.lessons.v1	3	∞	compact	Lessons learned — compacted, always available
ops.notifications.v1	3	7 d	delete	Outbound alerts (Slack, ITSM, email)
demo.payments.events	24	3 d	delete	Demo workload — high partition count for lag simulation

SCENARIO COVERAGE

11 Operational Scenarios

Covering consumer, producer, controller, schema, replication, and storage failure modes.

Consumer Lag Spike
KIP-848 | Gate
Requires approval

KRaft Controller Failover
KRaft | Auto
Autonomous remediation

Share Group Rebalance
KIP-932 | Gate
Requires approval

False-Positive Suppression
KIP-848 | Auto
Autonomous remediation

Schema Registry Mismatch
Avro | Gate
Requires approval

Broker Disk Saturation
I/O | Auto
Autonomous remediation

Under-Replicated Partitions
ISR | Gate
Requires approval

Producer Timeout Storm
Batch | Auto
Autonomous remediation

Consumer Session Timeout
GC | Auto
Autonomous remediation

Log Compaction Lag
Compact | Auto
Autonomous remediation

Partition Imbalance
Leader | Auto
Autonomous remediation

Gate = human approval required before execution

Auto = Monitor agent acts without intervention

POLICY GATE

Human-in-the-Loop Approval

No cluster-mutating operation executes without operator review. Every decision is recorded in ops.actions.audit.v1.

1. Anomaly detected
2. Root-cause analysis run
3. MCP tool call proposed
4. Execution paused at gate
5. Operator reviews context
6. Approve or Reject — both paths audited

Gate modal shows:

- Proposed MCP tool call + parameters
- Rationale from Monitor agent
- Infrastructure mutation risk level
- Root Cause — highest-confidence finding
- Available Root Causes — ranked candidates, most probable highlighted
- Approve / Reject — No Action

Structured Incident Analysis

Each scenario produces a ranked set of candidate causes alongside the actual finding

Example — Consumer Lag Spike on payments-consumer-group

ROOT CAUSE

Consumer group processing rate fell below producer write rate for a sustained period.

AVAILABLE ROOT CAUSES

1. **Consumer group processing rate fell below producer write rate for a sustained period.**
2. Downstream dependency slowdown (database or I/O bottleneck) caused consumer thread stalls.
3. Mid-consumption partition rebalance interrupted offset progress.
4. Topic partition count insufficient for current consumer group size.

OBSERVABILITY

Audit Trail, Lessons, and Real-Time Visibility

Audit Log

- Every agent action recorded with timestamp and actor
- Tool call parameters and execution result
- Approval and rejection decisions attributed by identity
- Produce and consume events across all 7 topics
- Persisted to ops.actions.audit.v1 (365-day retention)

Lessons Learned

- Recorded after each resolved scenario
- Includes: action taken, outcome, lag delta
- Adjusted threshold recommendation
- Effectiveness rating (Effective / Needs Review)
- Written to ops.lessons.v1 — log-compacted, no expiry

Live Event Stream

- Server-Sent Events (SSE) feed from the server
- Drives the animated agent mesh canvas in the UI
- Per-topic health: lag, offset, partition count
- Consumer group tracking per scenario
- Broker topology: epoch, ACLs, mTLS status

ops.lessons.v1 is log-compacted — lessons are never expired, and the Monitor agent reads them on every startup cycle.

TECHNICAL STACK

Implementation — POC Stack

Framework	Next.js 14 (App Router) · TypeScript 5 · React 18
Kafka Client	KafkaJS · Admin API · Producer / Consumer
Real-Time UI	Server-Sent Events (SSE) · React Flow (agent canvas)
Agent Tooling	MCP (Model Context Protocol) · JSON-RPC tool registry
Cluster	Confluent for Kubernetes (CFK) 8.2 . Kafka . KRaft · DigitalOcean Bootstrap
State	React state (client UI) · Kafka topics (persistent) · localStorage (lesson cache)
Auth	NextAuth v4 · Google OAuth
Build & Deploy	Vercel (Free tier, Cloud Platform, self managed) · Public GitHub CI · Auto-deploy on push

POC scope: single-process deployment. A production version would deploy each agent as an independent service with dedicated consumer